

Migration Legacy Access – PostgreSQL

Mapping complet des champs source – destination

Generé le 26 juin 2026

Architecture globale

1. **Sources** : Deux bases MS Access exportées en SQL via `mdb-export` dans des schémas de staging :
 - `legacy_fsr` : Base Factures des Fournisseurs.accdb
 - `legacy_clients` : Base Factures Clients.accdb
2. **Staging** : Chargé dans PostgreSQL via `make migrate-legacy-load` (INSERT avec ON CONFLICT DO NOTHING).
3. **Migrateur** : `php artisan legacy:migrate` – lecture du staging, insertion dans les tables applicatives via Eloquent.
4. **Post-traitement** : `recomputeSoldes()` – recalcul SQL de `montant_paye`, `reste_a_payer`, `statut`, `date_solde`.

Ordre d'exécution

#	Etape
1	Plan comptable (<code>listcompt</code> , <code>compteniv1-9</code> , <code>classecompte</code>)
2	Fournisseurs
3	Clients
4	Banques + Comptes bancaires + Bordereaux/Approvisionnements
5	Factures fournisseurs
6	Factures clients
7	Reglements fournisseurs
8	Reglements clients
9	Imputations + Ecritures comptables
10	Utilisateurs
11	<code>recomputeSoldes()</code> (toujours execute)

Conventions de couleur

Vert	Champ source (staging Access)
Bleu	Champ destination (table applicative)
Orange	Transformation / logique speciale

Etape 1 : Plan comptable

Source : `legacy_clients.listcompt`

Champ source	Champ dest.	Transformation
numcmpt	numero_compte	Nettoye (<code>clean()</code>) : trim + suppression BOM
libelle	libelle	Tronque a 255
classe	classe	1er chiffre du numero ou de la classe
—	niveau	Calcule : <code>min(max(len-1, 1), 5)</code>
—	is_custom	Toujours true

Source : `legacy_fsr.compteniv1-9`

Champ source	Champ dest.	Transformation
numcmpt{N}	numero_compte	Cle primaire de chaque niveau
intitule	libelle	—
—	classe	1er chiffre du numero
—	niveau	Niveau N

Source : `legacy_fsr.classecompte`

Champ source	Champ dest.	Transformation
numclass	code	Cle
intitule	libelle	—
—	prefixe_compte	= code
—	is_active	Toujours true

Etape 2 : Fournisseurs

Table source : `legacy_fsr.fournisseur`

Table dest. : `fournisseurs`

Cle : `nom` (`updateOrCreate`)

Champ source	Champ dest.	Transformation
rsfsr	nom	<code>clean()</code> + tronque a 255
comptfsr	compte_comptable_id	Lookup via <code>ensureCompte()</code>
numifu	ifu	Extraction chiffres, 8-13 caracteres
numtel	telephone	Tronque a 30
pays	pays	Conversion nom/code ISO 2 lettres
ville	ville	Tronque a 100
contact	contact	Tronque a 255
info	observations	<code>clean()</code>
—	type_fournisseur	Toujours Autres
user	—	Ignore

Note : Le cache `fournisseurByCompte` est peuple par `comptfsr` et par `NOM:rsfsr`. La facture utilise ce cache pour retrouver le fournisseur via `comptimp` ou `rsfsr`.

Etape 3 : Clients

Table source : `legacy_clients.client`

Table dest. : `clients`

Cle : (`compte_comptable_id`, `nom`)

Champ source	Champ dest.	Transformation
numcli	compte_comptable_id	<code>str_replace(',', '.', ...)</code> puis <code>ensureCompte()</code>
rscli	nom	Tronque a 255, fallback Client + numcli
adrcli	adresse	<code>clean()</code>

—	type_client	Toujours divers
---	-------------	-----------------

Etape 4 : Banques / Comptes bancaires / Bordereaux

Banque → banques

Champ source	Champ dest.	Transformation
libbanqdep	nom	firstOrCreate, fallback = cptbanqdep
cptbanqdep	—	Utilise pour le compte bancaire

Compte bancaire → comptes_bancaires

Champ source	Champ dest.	Transformation
cptbanqdep	numero_compte	updateOrCreate
—	banque_id	FK vers la banque creee
—	solde	Toujours 0

Bordereau → approvisionnements_banques

Champ source	Champ dest.	Transformation
cptbanqdep + refbordep	compte_bancaire_id + reference_bordereau	Cle composite
datdep	date_depot	date() → YYYY-MM-DD
mtdep	montant	num() → float
observ	observations	—

Approvisionnement fournisseur

Cote fournisseurs, pas de table banque. Le migrateur cree une banque generique.

Champ source	Champ dest.	Transformation
numcmptbanq	compte_bancaire_id	Recherche ou creation
numdep	reference_bordereau	clean()
datedep	date_depot	date()
mtdep	montant	num()
observ	observations	—

Etape 5 : Factures fournisseurs

Table source : legacy_fsr.facture

Table dest. : factures_fournisseurs

Cle : numero_piece (firstOrNew)

Champ source	Champ dest.	Transformation
nump	numero_piece	clean() + tronque a 50
datenreg	date	Date PC = date d'enregistrement. saneDate() rejette annees < 2000 ou > courante+1. Repli : datfac puis annee puis date brute
datfac	date_facture_bc	Date facture/BC = date inscrite par le fournisseur. Repli : datenreg
reffac	reference_facture	Tronque a 100
comptimp	fournisseur_id	Lookup cache fournisseurByCompte[comptimp], repli NOM:rsfsr
rsfsr	fournisseur_nom	Copie du nom (pas FK)
libfac	libelle	Tronque a 500
mtfac	montant_facture	num() → float
mtmd	montant_mo	num() → float
avoir	avoir	num() → float
tacpt	taux	Attention : taux AIB (pas taib = 0)
mtacpt	montant_reduction	Montant AIB (pas mtaib)
numcptacpt	type_reduction	Compte comptable de l'AIB
statu	date_solde	Si true : saneDate(datreg) repli saneDate(datenreg) repli date PC. Sinon null
user	created_by_name	—
tesc	—	Ignore (TVA = 0)
taib	—	Ignore (= 0, utiliser tacpt)
mtaib	—	Ignore (utiliser mtacpt)
datreg	—	Utilise uniquement pour date_solde
numord	—	Ignore
annee	—	Repli pour date_pc
etatimp	—	Ignore
—	assujetti_tva	Toujours false
—	statut	Toujours validee

Regles critiques pour les dates :

- date (PC) = datenreg (date d'enregistrement)
- date_facture_bc = datfac (date inscrite par le fournisseur)
- Dates aberrantes (< 2000 ou > annee courante+1) rejetees
- Repli : datenreg → datfac → annee (1er jan) → date brute

Etape 6 : Factures clients

Table source : legacy_clients.facture

Table dest. : factures_clients

Cle : reference (firstOrNew)

Champ source	Champ dest.	Transformation
reffac	reference	clean() + tronque a 20
datfac	date_facture	saneDate() repli datregfac repli ann (1er jan) repli date brute
mtfac	montant	num() → float
numcli	client_id	str_replace(',', '.', ...) puis lookup clientByNumcli
etatfac	date_solde	Si true : saneDate(datregfac) repli date_facture
user	created_by_name	—
datregfac	—	Utilise pour date_solde
ann	—	Repli pour date_facture
—	statut	Toujours non_payee

—	client_nom	Copie depuis le client trouve
---	------------	-------------------------------

Etape 7 : Reglements fournisseurs

Table source : legacy_fsr.reglement

Table dest. : reglements_fournisseurs

Cle : (facture_id, date_reglement, montant, reference)

Champ source	Champ dest.	Transformation
nump	facture_id	Lookup via cache factureFsrByNumP
datreg	date_reglement	date() → YYYY-MM-DD
mtreg	montant	num() → float
numch	reference	Tronque a 100
modreg	mode_paiement	Mapping : especes/virement/mobile_money/carte/cheque
insreg	beneficiaire	Tronque a 255, fallback = nom fournisseur
raib	deduire_aib	bool : true si > 0
lreg	numero_ligne	Tronque a 50
user	created_by_name	—
—	numero_reglement	Hash MD5 : LEG/ + 16 hex
—	fournisseur_id	Copie depuis la facture
—	fournisseur_nom	Copie depuis la facture
—	facture_numero	Copie depuis la facture
—	statut	Toujours valide
comptinsreg	—	Ignore
datch	—	Ignore
exereg	—	Ignore
ord	—	Ignore
impr	—	Ignore
mtregl	—	Ignore (montant en lettres)

Etape 8 : Reglements clients

Table source : legacy_clients.reglement

Table dest. : reglements_clients

Cle : (facture_id, date_reglement, montant, reference_cheque)

Champ source	Champ dest.	Transformation
reffac	facture_id	Lookup cache factureClientByRef. BOM cleanup avant lookup
datreg	date_reglement	date()
mtch	montant	num()
refch	reference_cheque	Tronque a 100
lreg	numero_ligne	Tronque a 50
insreg	institution	Tronque a 255
cptbanqdep + refbordep	approvisionnement_id + banque_depot_id	Rattachement bordereau si les 2 existent et != 0
user	created_by_name	—
—	client_id	Copie depuis la facture
—	client_nom	Copie depuis la facture
—	facture_reference	Copie depuis la facture
—	type_reglement	Toujours reglement
numch	—	Ignore
datdep	—	Ignore
mtdep	—	Ignore

Etape 9 : Imputations + Ecritures comptables

Table source : legacy_fsr.imputation

Tables dest. : imputations_facture_fournisseur + ecritures_comptables

Imputation facture

Champ source	Champ dest.	Transformation
nump	facture_id	Lookup via cache ou DB
numcpt	compte_id	str_replace(',', '.', ...) puis ensureCompte()
debit vs credit	nature	debit $\geq$ credit ? debit : credit
debit credit	montant	max(debit, credit)
libelle	libelle	clean()
numcpt vide	—	Ligne ignoree

Ecriture comptable

Champ source	Champ dest.	Transformation
nump	facture_id	Lookup
numcpt	numero_compte	Chaine brute (pas de FK)
debit	debit	num()
credit	credit	num()
libelle	libelle	clean()
datimp	date_ecriture	date()
lreg	type	Si renseigne : reglement, sinon facture

Backfill compte_id des factures

Après les imputations, le migrateur renseigne `factures_fournisseurs.compte_id` :

- Cherche les imputations DEBIT dont le compte commence par 6, 42 ou 2
- Prend le montant le plus eleve
- Ne touche que les factures dont `compte_id` est NULL

Etape 10 : Utilisateurs

Sources : `legacy_clients.user`, `legacy_clients.user 1`, `legacy_fsr.user`

Table dest. : `users`

Champ source	Champ dest.	Transformation
nompre / nomuser / pseudo	name	clean()
—	email	slug(name)@legacy.local
—	password	Hash de <code>Str::random(24)</code> (aucun mdp herite)
—	is_active	Toujours false
—	role	Toujours <code>ROLE_USER</code>
typuser / service	poste	clean()
civilite	—	Ignore
operation	—	Ignore
pass	—	Ignore (non migre par securite)

Etape 11 : RecomputeSoldes()

Execute a chaque fin de migration (2 requetes groupees SQL).

Factures clients

Champ calcule	Formule
montant_paye	SUM(reglements_clients.montant) WHERE type_reglement='reglement' AND deleted_at IS NULL
reste_a_payer	GREATEST((montant - ristourne) - montant_paye, 0)
statut	Si montant_paye $\leq$ 0 : non_payee. Si montant_paye + 0.01 $\geq$ montant : payee. Sinon : partiellement_payee

Puis :

- Facture avec `date_solde NOT NULL` : statut = payee, reste_a_payer = 0

- Facture payee sans date_solde : `date_solde = MAX(date_reglement)`
- Dernier repli : payee sans date_solde ni reglement : `date_solde = date_facture`

Factures fournisseurs

Champ calcule	Formule
montant_paye	SUM(reglements_fournisseurs.montant) WHERE statut != annule AND deleted_at IS NULL
reste_a_payer	GREATEST(montant_net - montant_paye, 0)
statut	Si montant_paye ≤ 0 : validee. Si montant_paye + 0.01 ≥ montant_net : payee. Sinon : partiellement_payee

Memes passes de `date_solde` que cote clients.

Fonctions d'aide (helpers)

Helper	Description
<code>clean(\$v)</code>	Trim + suppression BOM UTF-8 (U+FEFF) + null si vide
<code>cut(\$v, \$max)</code>	<code>clean()</code> + <code>mb_substr</code>
<code>num(\$v)</code>	Cast float, suppression de tout sauf chiffres/points/tirets
<code>date(\$v)</code>	Carbon ::parse() → YYYY-MM-DD, null si echec
<code>saneDate(\$v)</code>	Comme <code>date()</code> mais rejette annees < 2000 ou > courante+1
<code>yearToDate(\$annee)</code>	Reconstruit 1er janvier d'une annee plausible
<code>bool(\$v)</code>	Interprete t/1/true/oui/yes/y
<code>paysCode(\$v)</code>	Nom pays → code ISO 2 lettres (default : BJ)
<code>mode(\$v)</code>	Mapping mode reglement (default : cheque)
<code>key(\$v)</code>	<code>strtolower(trim(...))</code> – normalisation caches
<code>ensureCompte()</code>	<code>firstOrCreate</code> par <code>numero_compte</code>

Problemes connus et pieges

1. **BOM UTF-8** : `mdb-export` place un U+FEFF en tete de chaque table. `clean()` doit etre appele **avant** `key()` partout.
2. **Doublons mdb-export** : certaines tables generent des lignes en double. Contraintes UNIQUE + ON CONFLICT DO NOTHING.
3. **Virgules Access** : Access stocke 4241,094 au lieu de 4241.094. Normalisation `str_replace` sur `numcli` et `numcpt`.
4. **AIB vs tacpt** : `taib/mtaib` = 0 quasi toujours. Le vrai AIB est dans `tacpt/mtacpt`.
5. **Fournisseur sans compte** : `comptfsr` NULL = pas de rattachement = absent du rapport par compte.
6. **Factures orphelines** : `numcli` inconnu dans le cache = facture client ignoree.
7. **Reglements orphelins** : facture inconnue = reglement ignore.
8. **Solde -0** : `max(0, round(-0.0001, 2))` peut donner -0. Corrige par `if ($solde < 0) $solde = 0`.